

Parameter-Efficient Fine-Tuning for Large Models: A Comprehensive Survey

Zeyu Han¹, Chao Gao², Jinyang Liu¹, Jeff (Jun) Zhang³, and Sai Qian Zhang⁴

¹Northeastern University ²University of California, Riverside ³Arizona State University
⁴New York University

{han.zeyu, liu.jinyang}@northeastern.edu, cgao037@ucr.edu, jeffzhang@asu.edu, sai.zhang@nyu.edu

Abstract—Large models represent a groundbreaking advancement in multiple application fields, enabling remarkable achievements across various tasks. However, their unprecedented scale comes with significant computational costs. These models, often consisting of billions of parameters, require vast amounts of computational resources for execution. Especially, the expansive scale and computational demands pose considerable challenges when customizing them for particular downstream tasks, particularly over the hardware platforms constrained by computational capabilities.

Parameter Efficient Fine-Tuning (PEFT) provides a practical solution by efficiently adapt the large models over the various downstream tasks. In particular, PEFT refers to the process of adjusting the parameters of a pre-trained large models to adapt it to a specific task or domain while minimizing the number of additional parameters introduced or computational resources required. This approach is particularly important when dealing with large-scale language models with high parameter counts, as fine-tuning these models from scratch can be computationally expensive and resource-intensive, posing considerable challenges in the supporting system platform design.

In this survey, we present comprehensive studies of various PEFT algorithms, examining their performance and computational overhead. Moreover, we provide an overview of applications developed using different PEFT algorithms and discuss common techniques employed to mitigate computation costs for PEFT. In addition to the algorithmic perspective, we overview various real-world system designs to investigate the implementation costs associated with different PEFT algorithms. This survey serves as an indispensable resource for researchers aiming to understand both the PEFT algorithm and its system implementation, offering detailed insights into recent advancements and practical applications.

Index Terms—Large Language Model, Parameter-Efficient Fine-tuning, Computer System, Distributed System.

I. INTRODUCTION

Large Models (LMs) have recently captured considerable public interest. Their ability to understand context and nuances enables them to proficiently handle diverse tasks across multiple domains, including natural language processing (NLP), computer vision (CV), etc. In the field of NLP, Large Language Models (LLMs) have achieved significant advancements across various tasks including text generation [10], [243], translation [239], [61], personalized chat-bots [192], [103], [187], and summarization [212], demonstrating remarkable proficiency.

Earlier studies [10] has suggested that LLMs exhibit high levels of generalization, enabling them to apply their acquired

knowledge to new tasks not included in their original training. This capability is commonly known as *zero-shot learning*. Nevertheless, fine-tuning remains essential to further enhance LLMs for optimal performance on new user datasets and tasks.

Due to its scale, a widely adopted strategy for fine-tuning LLMs involves adjusting a limited number of LLM parameters while keeping the remainder unchanged. This technique, termed *Parameter-Efficient-Fine-Tuning* (PEFT), involves selectively adjusting a small proportion of their parameters, while keeping the rest unaltered. Furthermore, the application of PEFT extends beyond the realm of NLP and quickly attracts interest in the CV community for handling fine-tuning vision models with large parameters, such as Vision Transformers (ViT) and diffusion models, as well as disciplinary models such as vision-language models.

In this survey, we systematically review and categorize recent advancements in PEFT algorithms as well as the system implementation costs associated with various PEFT algorithms across diverse scenarios. Figure 1 presents the overview content for this survey. In section II, we present some fundamental concepts for LLM and PEFT, including computational flow for LLM, basic knowledge of PEFT, and commonly used datasets and tasks.

We categorized all types of PEFT algorithms in Section III according to their computational flow. In Section III-A, we introduce additive algorithms that either introduce additional weight parameters or modify activations. For algorithms that exclusively require fine-tuning with existing parameters, they fall under the category of selective approaches, and their introduction can be found in Section III-B. In Section III-C, we explore reparameterized PEFT, which constructs a (low-dimensional) reparameterization of original model parameters for training while transforms the weights back to maintain the inference speed. Additionally, there exist algorithms that combine the above techniques, and we have classified these as hybrid approaches, elaborating on them in Section III-D. We also investigate strategies for further reducing computational complexity of different PEFT algorithms, including KV-cache management, pruning, quantization, and memory optimization, in Section IV.

In Section V, we expand the scope of this survey beyond the computational perspective to involve various potential application scenarios. We explore innovations that applying PEFT techniques to different model architecture, including LLMs

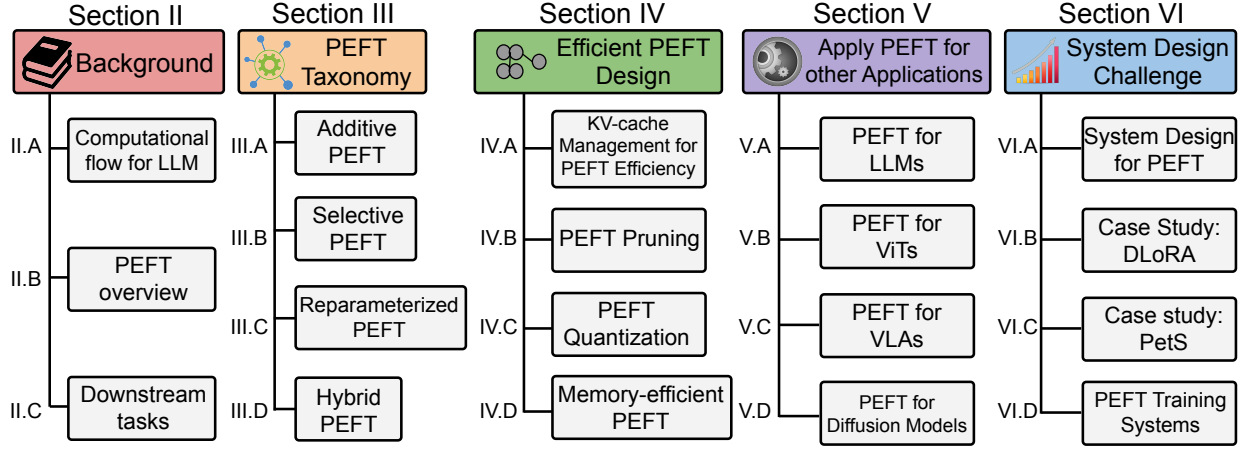


Fig. 1: A content overview covered in the survey.

(Section V-A), Vision Transformer (Section V-B), Vision-Language alignment models (Section V-C), and Diffusion models (Section V-D), for varied downstream tasks, underscoring PEFT's versatility and applicability in a range of scenarios.

In Section VI, we explore the system design challenge for PEFT methods. The discussion includes three advanced system solutions for practical PEFT deployment: distributed tuning (Section VI-B), PEFT query serving (Section VI-C), and concurrent PEFT tuning (Section VI-D).

In the last Section VII, we summarize our survey and propose several potential future directions from both algorithm and system perspectives, hoping to provide valuable insights for further research and development in the field.

II. BACKGROUND

In this section, we first discuss the computation flow of LLM, including its fundamental components, computational complexity, and the flow of computations it involves as a case study. We then provide a brief overview of different PEFT algorithms in section II-B.

A. Computation flow for LLaMA

In order to gain a deeper understanding of LLM and other Transformer-based models, we employ LLaMA-7B, a cutting-edge open-source LLM model, to scrutinize the architecture of LLM as well as Transformer. As shown in Figure 2 (a), LLaMA consists of three major components: an embedding block, a stack of decoder blocks and a head block which consists of linear and softmax layer. The embedding layer's primary role is to transform unstructured textual information, into chunks of discrete numerical vectors (*tokens*) to facilitate subsequent processing. The embedded tokens are then delivered to the decoder layers for further processing. Each LLaMA decoder is composed of two fundamental components: Multi-head Self-Attention (MSA) and Feedforward Network (FFN). In the MSA module, each of the tokens will be clustered by an attention map obtained by a dot production between two linear mappings of the input tokens. Then the grouped tokens will be further processed by a Feedforward Neural network.

Additionally, Root Mean Square Layer Normalization (RMSNorm) [211] is adopted in LLaMA as a replacement for Layer Normalization to ensure efficient training.

LLM distinguishes itself from other deep neural network (DNN) models such as convolutional neural networks (CNN) in two significant ways. Firstly, LLM exhibits an inherent autoregressive nature, necessitating multiple iterations to complete the generation task. Moreover, LLM incorporates an attention mechanism, a component with computational complexity that scales quadratically with the length of the inputs. On the other hand, the inherent computation characteristic of LLM lies in the attention blocks inside each decoder layer. Figure 2 (c) depicts the high-level overview of the computation flow in the attention block.

During the inference process, each decoder takes a 4-dimensional tensor $x \in \mathbb{R}^{b \times l \times d}$ as the input tokens. The input tokens are first multiplied with three weight matrices W_Q , W_K , and W_V , producing the output referred to as query(Q), key(K) and value(V). Given the MSA module's inability to recognize positional data and the inherent autoregressive nature of LLMs, the query and key will undergo a process using Rotary Positional Embedding [165] (RoPE, denoted as $R(\cdot)$ in Eq 1) to encode the position information. Subsequently, the key and value will be combined with prior tokens.

After the positional embedding, the intermediate activation will then undergo a series of multiplication, softmax, and residual addition to generate MSA output as described in Eq 9. To be noted here, d_k in the equation refers to the number of feature dimensions in the multi-head attention mechanism.

$$Q, K, V = R(W_q x), R(W_k x), W_v x \quad (1)$$

$$SA(x) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_{head}}}\right)V \quad (2)$$

$$MSA(x) = [SA_1(x); SA_2(x); \dots; SA_k(x)]W_o \quad (3)$$

The SA output will then be forwarded to the FFN blocks for further processing. The FFN block will have another three

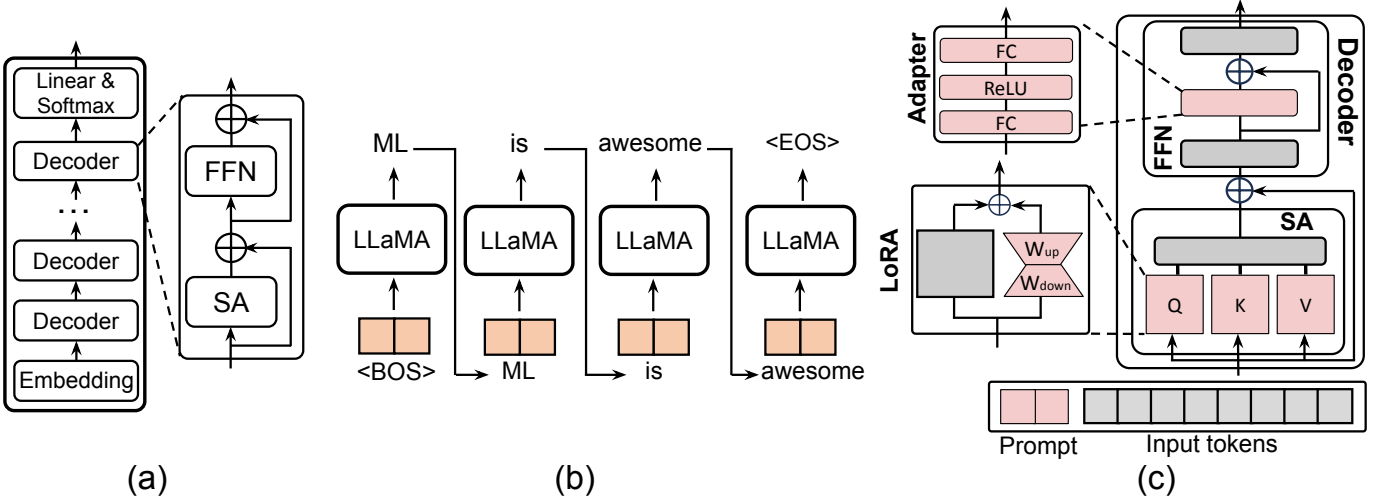


Fig. 2: (a) LLaMA architecture. (b) LLaMA auto-regressive pattern. (c) Three common PEFT operations. All the learnable components are highlighted in red, while the frozen components are highlighted in grey. LoRA is applied on all the Query, Key, and Value blocks. The adapter targets the FFN module. Soft-Prompt focused on tuning the input activation of each decoder. We only show one decoder for illustration simplicity.

matrices W_{up} , W_{down} , and W_{gate} and the computation can be illustrated by:

$$FFN_{LLaMa}(x) = W_{up}(SiLU(W_{gate}x) \odot (W_{down}x)) + x, \quad (4)$$

where x denotes the input of the FFN layer, and $SiLU$ is the nonlinear function used in LLaMA. In the original Transformer, the FFN block can be demonstrated by:

$$FFN_{Transformer}(x) = W_{up}(ReLU(W_{down}x)) + x. \quad (5)$$

The output of the last decoder layer will be sent to a linear layer, which then generates a probability distribution spanning the complete *vocabulary* to predict the next token in the sequence. The produced token will then be concatenated with the previous tokens and used as the input for the next round of processing. This generating process repeats in an auto-regressive manner until a full sequence of tokens, referred to as a *completion*, is produced (Figure 2 (b)). For training, the computation flow is similar to that for inference, except that the generated sentences are directly compared to the ground truth output and generate the training loss. Gradients will then be computed across the LLM weights to minimize this training loss.

To analyze the computation cost and memory overhead in LLM, we also set a series of parameters used in later section III. Table I shows the parameter size and computation dimension in the LLaMA-7B model as a starting example.

LLM models generate tokens (words) one for each round, depicted in Fig 2, based on the previous prompt (input) and previously generated sequence. This process will be repeated until the model outputs hits a termination token. To accelerate the inference process in LLM models, people take the strategy of storing the previous Keys and Values in Key-Value cache (KV-cache), so they don't need to recalculate them for each new token. Mathematically, we can represent the total decoders' kv-cache memory cost in equation 6. In

the equation, l and b are the context length and batch size and L refers to the number of layers. The d_{head} is the head dimension and n_{head} is the number of heads.

$$Size = L \times 2 \times b \times l \times d_{head} \times n_{head} \quad (6)$$

B. Overview on Parameter Efficient Fine Tuning

Fine-tuning remains essential to enhance LLM performance on unseen user datasets and tasks. With the size of the model growing (e.g. 1.5B in GPT-2 to 175B in GPT-3), standard full fine-tuning paradigm requires thousands of GPU work in parallel, which is highly inefficient and unsustainable. A type of algorithm has been raised namely Parameter-efficient fine-tuning (PEFT) which aims to tune minimal parameters to achieve better performance over full tuning on downstream tasks.

In parallel developments, large-scale pre-trained models in vision and multimodal domains have also demonstrated their effective representational learning capabilities, enabling adaptation from large datasets to smaller ones or across various data modalities through fine-tuning. Consequently, this capability has made PEFT increasingly attractive to the wider research community.

We categorized the PEFT algorithms into **additive**, **selective**, **reparameterized**, and **hybrid** fine-tuning based on their operations. As Figure 3 depicts, three major **additive fine-tuning** algorithms are normally used: (1) Adapter; (2) Soft Prompt; (3) Others. They differ from each other in terms of the different additional tunable modules or parameters. **Selective fine-tuning**, on the other hand, doesn't require any additional parameters, it selects a small subset of parameters from the backbone model and only makes them tunable while keeping the majority of parameters untouched during fine-tuning on downstream tasks. We categorized selective fine-tuning based on the grouping of chosen parameters: (1) Unstructural Masking; (2) Structural Masking. Reparametrization

Operation	Weights Symbol	Weights Dimension	Input Tensor Dimension	Complexity
Eq. 1	W_Q, W_K, W_V	$d \times k \times \frac{d}{k}$	$b \times l \times d$	$O(l)$
Eq. 2	-	-	$b \times l \times 3 \times k \times \frac{d}{k}$	$O(l^2)$
Eq. 3	W_o	$d \times d$	$b \times l \times d$	$O(l)$
Eq. 4	$W_{up}, W_{down}, W_{gate}$	$d \times 4d$	$b \times l \times d$ OR $l \times b \times 4d$	$O(l)$

TABLE I: Configuration parameters and computation operation for LLaMA-7B architecture

represents transforming model parameters between two equivalent forms. Specifically, **reparametrized fine-tuning** introduces additional low-rank trainable parameters during training, which are then integrated with the original model for inference. This approach is categorized into two main strategies: (1) Low-rank Decomposition, and (2) LoRA Derivatives. **Hybrid fine-tuning** explores the design spaces of different PEFT methods and combines their advantages.

C. Downstream Tasks for LLM Evaluation

Two types of tasks have been widely used for LLM evaluation, the first type is the General Language Understanding Evaluation (GLUE) [175] benchmark, which integrates nine sentence or sentence-pair language understanding tasks (CoLA, SST-2, MRPC, STS-B, QQP, MNLI, QNLI, RTE, and WNLI), chosen for their diversity in dataset sizes, text genres, and difficulty levels, and is based on established existing datasets. It also includes a diagnostic dataset specifically designed to evaluate and analyze model performance across various linguistic phenomena inherent in natural language. Additionally, it features a public leaderboard to track performance on the benchmark and a dashboard to visualize model performance on the diagnostic set.

The other type of dataset that has been used in recent LLM papers is common sense reasoning which integrated into our study caters to a variety of research facets: (1) *OpenBookQA* [135] is curated to foster research in advanced question-answering, delving into a profound understanding of both the subject matter and the language in which it is articulated. (2) *PIQA* [8] primarily emphasizes everyday scenarios, demonstrating a predilection for unconventional solutions. (3) *Social IQA* [157] emerges as a novel question-answering benchmark tailored for gauging social common-sense intelligence. (4) *HellaSwag* [208] serves as a dataset, the essence of which is to ascertain the capability of machines in aptly concluding sentences. (5) *BoolQ* [30] is a dataset dedicated to question-answering, particularly for binary responses (yes/no queries). (6) *WinoGrande* [156] is introduced as a fresh compilation, encompassing a substantial 44,000 problems. (7) *ARC-easy* [31] presents itself as a novel dataset constituting genuine grade-school level multiple-choice science questions, designed to invigorate research in intricate question-answering. (8) *ARC-challenges* [31], distinctively, encompasses solely those questions that were inaccurately addressed by both a retrieval-based algorithm and a word co-occurrence algorithm.

Image recognition is the primary benchmark and application for vision models, exemplified by benchmarks such as fine-

grained visual categorization (FGVC) and visual task adaptation benchmark (VTAB). Beyond image classification, video action recognition is another key application area, involving datasets like Kinetics-400 [92], SSv2 [56], and HMDB51 [96]. Additionally, PEFT has been utilized for dense prediction tasks, using datasets like MSCOCO [113], ADE20K [230], and PASCAL VOC [43].

III. PEFT TAXONOMY

The PEFT strategies can be broadly classified into four categories: **additive PEFT** (Section III-A), which modifies the model architecture by injecting new trainable modules or parameters; **selective PEFT** (Section III-B), which makes a subset of parameters trainable during fine-tuning; **reparameterized PEFT** (Section III-C), which constructs a (low-dimensional) reparameterization of the original model parameters for training, then equivalently transforms it back for inference; and **hybrid PEFT** (Section III-D), which combines advantages from different PEFT methods to build a unified PEFT model. A overview of different types of PEFT algorithms is depicted in Figure 4.

A. Additive PEFT

Standard full fine-tuning entails substantial computational expenses and also could potentially harm the model’s generalization ability. To mitigate this problem, a widely employed approach is to maintain the pre-trained backbone unchanged and introduce only a minimal number of trainable parameters that are strategically positioned within the model architecture. While fine-tuning for a specific downstream task, only the weights of these additional modules or parameters are updated, which results in a substantial reduction in storage, memory, and computational resource requirements. Due to their characteristic of adding parameters, these techniques can be termed as *Additive Tuning*, as shown in Figure 4 (a). Next, we discuss several popular Additive PEFT algorithms.

1) *Adapters*: Adapter approaches involve the insertion of small adapter layers within Transformer blocks. Typically, an adapter layer consists of a down-projection matrix $W_{\text{down}} \in \mathbb{R}^{r \times d}$, followed by a non-linear activation function $\sigma(\cdot)$, and an up-projection matrix $W_{\text{up}} \in \mathbb{R}^{d \times r}$. In this context, d represents the dimension of the hidden layer, and r serves as the bottleneck dimension, which is a hyperparameter used in configuring the adapters. Denote h_{in} as the input to the adapter, the computation within the adapter module (with residual) can be summarized as follows:

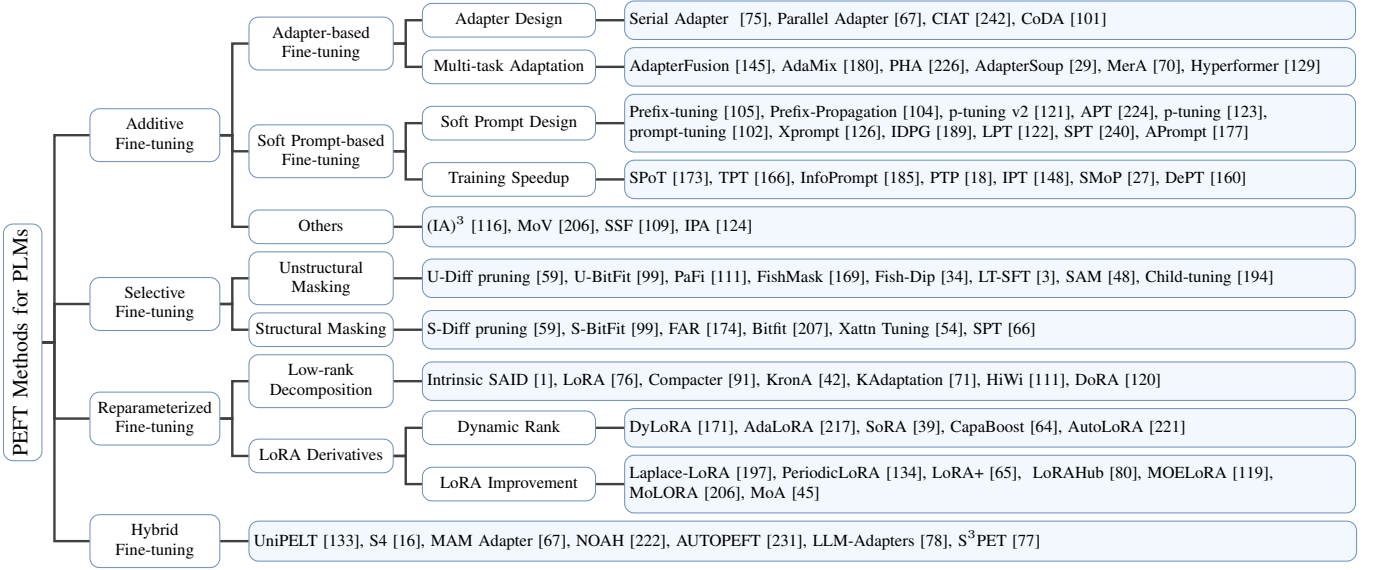


Fig. 3: Taxonomy of Parameter-Efficient Fine-Tuning Methods for Large Models.

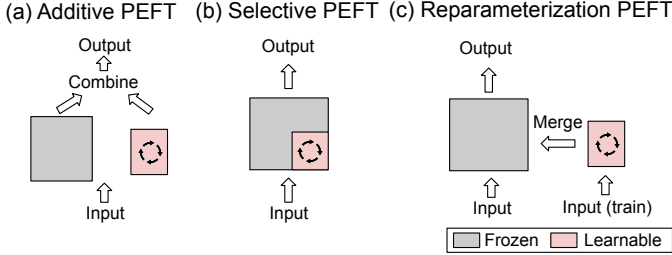


Fig. 4: Different types of PEFT algorithms.

$$\text{Adapter}(x) = W_{\text{up}}\sigma(W_{\text{down}}x) + x. \quad (7)$$

The concept of adapters in the field of NLP was initially introduced by **Serial Adapter** [75] as shown in Figure 5 (a). In their approach, each Transformer block is enhanced by adding two adapter modules, with one positioned after the self-attention layer and the other after the FFN layer, respectively. Subsequent research has aimed to address the additional computational cost associated with adapter layers. A modified framework **AdapterFusion** [145] was proposed, where adapter layers are inserted only after the 'Add & Norm' step following the FFN layer to enhance the computational efficiency. The adapters mentioned above follow a sequential design, placing adapter layers as bottlenecks within the Transformer blocks. This approach may potentially reduce the model's parallelism and require a trade-off between inference efficiency and accuracy. In contrast, [67] introduced a **parallel adapter** (PA) approach as depicted in Figure 5 (b), which reorganizes the traditionally sequential adapter layers into a parallel side-network that runs alongside each Transformer sublayer. Similarly, **CIAT** [242], **CoDA** [101] and **KronA** [42] also adopts a parallel adapter design. Except for the parallel design, **CoDA** employs a sparse activation mechanism to improve the inference efficiency as shown in Figure 5 (c). Specifically, CoDA use a soft top- k selection process that

identifies k important tokens in each layer, which will be processed by both the frozen pre-trained Transformer layer and the adapter branch to maintain model accuracy. In contrast, those unimportant tokens are only processed by the adapter branch while skip the heavy pre-trained layer, therefore optimizing for inference efficiency without compromising overall performance.

To enhance the performance and generalization of adapters, various studies have implemented *multi-task learning* strategies, such as **AdapterFusion** [145], **AdaMix** [180], **PHA** [226], **AdapterSoup** [29], **MerA** [70], and **Hyperformer** [129]. **AdapterFusion** keeps all pre-trained adapters in the model and employs a fusion module to merge the multi-task informations. Unlike AdapterFusion, **MerA** merges pretrained adapters into a single one through optimal transport based on weights and activations. This approach avoids introducing any additional trainable parameters, thereby enhancing computational efficiency. **Hyperformer** stores the multi-task information in a shared hypernetwork, which generates task and layer-specific adapter parameters conditioned on task and layer id embeddings. Given a new task, only an additional task embedding needs to be learned, therefore reducing the number of trained parameters.

2) *Soft Prompt*: Alternatively, prompt tuning presents an additional approach for refining the model to achieve improved performance through fine-tuning. Instead of optimizing discrete token representations through in-context learning, there is a prevailing belief that the continuous embedding space of soft prompts inherently contains more information [144]. Drawing inspiration from this concept, researchers directly append adjustable vectors, referred to as soft prompts, to the start of the input sequence. This can be represented as follows:

$$\mathbf{X}^{(l)} = [\mathbf{s}_1^{(l)}, \dots, \mathbf{s}_{N_s}^{(l)}, \mathbf{x}_1^{(l)}, \dots, \mathbf{x}_{N_x}^{(l)}] \quad (8)$$

where $\mathbf{X}^{(l)}$ is the sequence of input tokens for layer l , including soft prompt tokens $\mathbf{s}_i^{(l)}$ followed by the original input

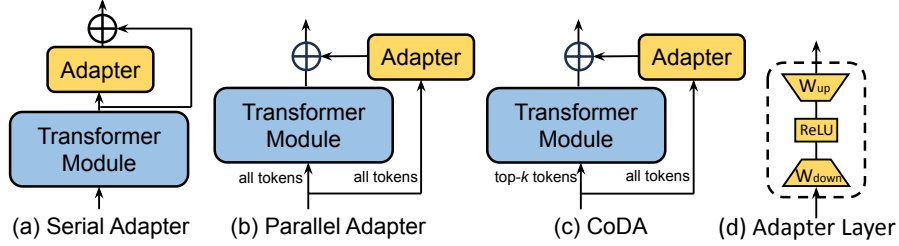


Fig. 5: Illustration of three representative adapter-based fine-tuning algorithms. Blue represents frozen, while yellow represents trainable.

tokens $\mathbf{x}_i^{(l)}$. N_S is the number of soft prompt tokens, and N_X is the number of original input tokens.

Prefix-tuning [105] introduces learnable vectors that are prepended to keys k and values v across all Transformer layers. To ensure stability during the optimization process, Prefix-tuning adopts a reparameterization strategy, which utilizes an MLP layer to generate these prefix vectors rather than optimizing them directly. After fine-tuning, only the prefix vectors are saved for inference. This technique is adapted and improved in several studies [104], [121], [224]. For instance, **p-tuning v2** [121] removes reparameterization and expands its usage to broader model scales and NLP tasks. **APT** (Adaptive Prefix Tuning) [224] enhances Prefix-tuning by introducing adaptive gate mechanism to control the prefix importance in each layer. Concurrent work **p-tuning** [123] and **prompt-tuning** [102] apply learnable vectors only at the initial word embedding layer rather than all layers to enhance training and inference efficiency. It's important to highlight that prompt-tuning demonstrates its effectiveness primarily in the context of large models, specifically those with over 11 billion parameters [102]. Complementing this, **Xprompt** [126] eliminates the negative prompt tokens through a hierarchical structured pruning, which closes the performance gap at smaller model scales. **The work by** [179] provides some theoretical analysis towards prompt tuning, demonstrating its universality and limitations in limited-depth Transformers. **IDPG** (Instance-Dependent Prompt Generation) [189] improves prompt tuning by generating prompts based on each input sentence with a lightweight prompt generator. In a related approach, **LPT** (Late Prompt Tuning) [122] also leverages a prompt generator to obtain instance-aware prompt. Unlike previous work, LPT adds these prompts only after an intermediate layer, rather than at the initial or all layers. This strategic placement eliminates the gradient calculation below the intermediate layer, thereby significantly accelerating the training speed. Simultaneously, LPT can improve the overall performance due to the shorter backpropagation path preserves more task-related information. Inspired by LPT, **SPT** (Selective Prompt Tuning) [240] delves deeper into the importance of prompt inserting strategies. It introduces a learnable probabilistic gate in each layer to determine whether to use the prompt propagated from the previous layer or inject a newly generated prompt. **APrompt** [177] employs another prompt inserting strategy. In addition to input prompts inserted at the beginning of the input sequence for each Transformer layer, APrompt also prepends additional learnable prompts to the respective query, key, and value

matrices in the self-attention blocks to learn new attention patterns. Besides, APrompt incorporates the learning of a task-specific head.

The concept of soft prompts has been employed for various downstream tasks [28], [184], although their training can be prone to instability and slow convergence. To address this, **SPoT** [173] uses a source prompt learned from one or multiple tasks to initialize prompts for new tasks. Similarly, transfer of soft prompts from one task to initialize another is proposed in **TPT** (transferable prompt tuning) [166], which demonstrates that a better prompt initialization results in a large training convergence speedup. **InfoPrompt** [185] develops two mutual information based loss functions, i.e., *head loss* and *representation loss*, to find better prompt initialization and learn sufficient task-relevant information, thereby also expediting convergence. **PTP** [18] delves into the root causes of training instability. It identifies the steep nature of the loss landscape in conventional prompt tuning, where minor variations in input data can lead to significant loss fluctuations. To mitigate this, PTP introduces perturbation-based regularizers to smooth the loss landscape and consequently stabilize the training process. **DePT** [160] decomposes the soft prompt into a shorter soft prompt with a pair of low-rank matrices, which are optimised with two distinct learning rates. This strategy not only improves the performance but also enhances training and inference efficiency. **SMoP** (Sparse Mixture-of-Prompts) [27] reduce the training and inference cost by utilizing short soft prompts. During training, multiple short soft prompts are trained, each tailored to specific subsets of the dataset. During inference, SMoP integrates a gating mechanism that routes each input instance to an appropriate short prompt. This technique not only increases efficiency in both training and inference stages but also retains performance comparable to those achieved with longer soft prompts. To further cut down the number of soft prompt parameters, **IPT** (Intrinsic Prompt Tuning) [148] identifies an intrinsic task subspace by training an auto-encoder on multiple tasks. Tuning on new tasks then requires adjusting only a few parameters within this subspace, significantly reducing the number of training parameters.

3) *Other Additive Methods*: Apart from the methods mentioned above, there appears other approaches that strategically incorporate additional parameters during the fine-tuning process. For example, **(IA)**³ [116] introduces three learnable rescaling vectors: $l_k \in \mathbb{R}^{d_k}$, $l_v \in \mathbb{R}^{d_v}$, and $l_{ff} \in \mathbb{R}^{d_{ff}}$, to rescale the key, value, and FFN activations, respectively, as depicted in Figure 6 (a). The operations within the self

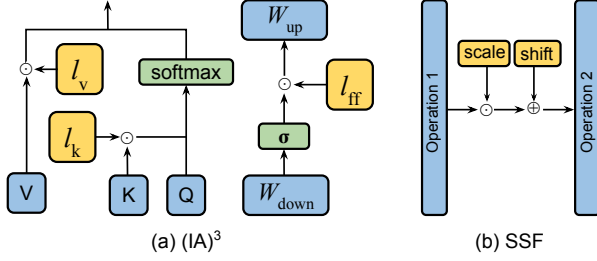


Fig. 6: Illustration of $(IA)^3$ and SSF. Blue represents frozen, while yellow represents trainable.

attention block can be described as follows:

$$SA(x) = \text{Softmax}\left(\frac{Q(l_k \odot K^T)}{\sqrt{d_{\text{head}}}}\right)((l_v \odot V). \quad (9)$$

In FFN, the rescaling can be denoted as:

$$FFN_{Transformer}(x) = W_{up}(l_{ff} \odot \sigma(W_{down}x)), \quad (10)$$

where $\odot$ is Hadamard product. Furthermore, the scale vectors l_k and l_v can be seamlessly integrated into the weight matrices of A_Q and A_W . This integration effectively eliminates the extra computational costs during inference. A similar technique **SSF** [109] also performs linear transformation to the model activations, as illustrated in Figure 6 (b). Specifically, after each operation (i.e., MSA, FFN and layer normalization) in the pre-trained model, a SSF-ADA layer is injected, which performs scaling and shifting to the features generated from the operation. During fine-tuning, only those SSF-ADA layers can be updated, while during inference, similar to $(IA)^3$, these SSF-ADA layers can be merged into model weights, so no additional inference overhead would be incurred. **IPA** (Inference-Time Policy Adapters) [124] offers a novel approach to align LLMs, such as GPT-4, with user-specific requirements without modifying the base model’s parameters. This is particularly significant when dealing with models whose parameters are extremely large and often not directly accessible. IPA achieves this by combining (through multiplication and normalization) the output distribution of a base LLM (base policy) with that of a smaller-sized model (adapter policy) during the decoding phase. During training, the policy adapter’s parameters are fine-tuned using reinforcement learning, while the base policy’s parameters remain fixed. During inference, IPA decodes with the combined distribution of the base model and the trained policy adapter, tailoring it to fulfill specific user-defined criteria.

B. Selective PEFT

Rather than additive PEFT, which increases the model complexity by adding more parameters, selective PEFT fine-tunes a subset of the existing parameters to enhance model performance over downstream tasks, as depicted in Figure 4 (b). Specifically, given a model with parameters $\theta = \{\theta_1, \theta_2, \dots, \theta_n\}$ where each θ_i denotes an individual model parameter and n represents the total count of these parameters, the process of selective PEFT is represented by applying a binary mask $M = \{m_1, m_2, \dots, m_n\}$ to these parameters. Each

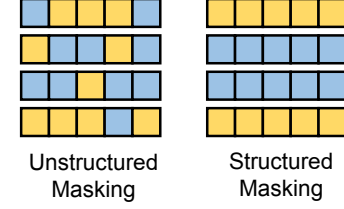


Fig. 7: Illustration of how two parameter masking methods. Blue represents frozen, while yellow represents trainable.

m_i in M is either 0 or 1, indicating whether the corresponding parameter θ_i is selected (1) or not selected (0) for fine-tuning. The updated parameter set θ' after fine-tuning is given by:

$$\theta'_i = \theta_i - \eta \cdot m_i \cdot \frac{\partial \mathcal{L}}{\partial \theta_i} \quad (11)$$

where η represents the learning rate, and $\frac{\partial \mathcal{L}}{\partial \theta_i}$ is the gradient of the loss function with respect to the parameter θ_i . In this formulation, only the parameters that are selected (i.e., $m_i = 1$) are updated during backpropagation.

Diff pruning [59] is a representative work that applies a learnable binary mask to the model weights during fine-tuning. To achieve parameter efficiency, the mask is regularized by a differentiable approximation of the L_0 -norm penalty. **PaFi** [111] simply select model parameters with the smallest absolute magnitude as trainable. **FishMask** [169] determines parameter importance using the approximate Fisher information. It then selects the top k parameters based on this information to form the mask M . Similarly, **Fish-Dip** [34] also uses Fisher information to calculate M , but the mask will be re-calculated dynamically in each train period. **LT-SFT** [3] introduces another technique to determines parameter importance inspired by the Lottery Ticket Hypothesis [46], [130], where the subset of parameters that change the most during an initial fine-tuning stage is selected to form the mask M . **SAM** [48] proposes a second-order approximation method, which approximates the original problem with an analytically solvable optimization function, to help decide the parameter mask. **Child-tuning** [194] proposes two approaches to select a child network during each training iteration, where only the parameters within this child network can be updated.

However, above unstructured parameter masking results in an uneven distribution of non-zero masks and diminished hardware efficiency when implementing PEFT. As shown in Figure 7, the structured mask organize parameter masking in regular patterns, unlike unstructured ones that apply it randomly, thus can enhances computational and hardware efficiency during training. Therefore, various structured selective PEFT techniques have undergone extensive investigation. **Diff pruning** proposes a structured pruning strategy by partitioning the weight parameters into local groups and strategically eliminating them together. Similarly, **FAR** [174] fine-tunes BERT models by grouping weights of the FFN in Transformer blocks into nodes, then rank and select the learner nodes using L_1 norm. To further reduce the memory access frequency, they also reconfigure the FFN by grouping the learner nodes together. **Bitfit** [207] is proposed to only fine-

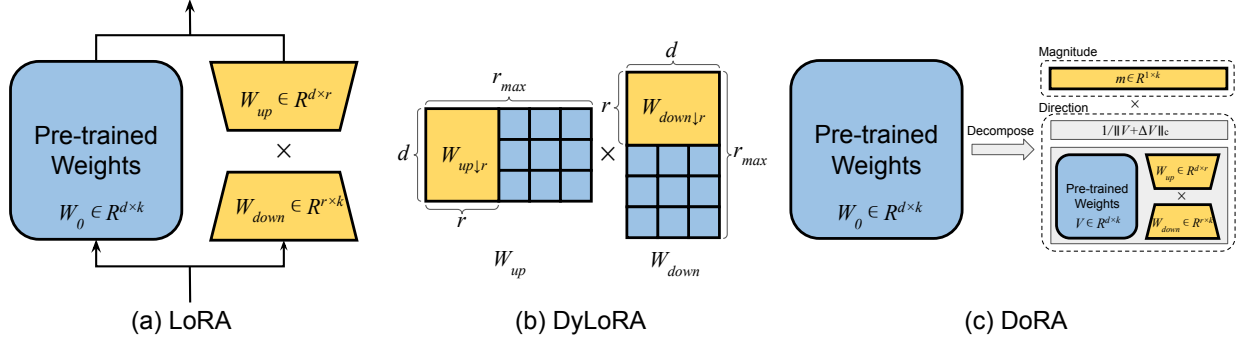


Fig. 8: Illustration of three representative reparameterized PEFT algorithms. Blue represents frozen, while yellow represents trainable.

tunes the bias parameters of each DNN layer, and achieves competitive results for small models. However, this method fails to handle large models. **The work by [99]** applies NAS to Bitfit, where **S-BitFit** keeps the structural nature in Bitfit that restrict NAS algorithm must choose whether $\delta b = 0$ or not for each bias module. Similar to Bitfit that fine-tunes a specific module in Transformer, **Xattn Tuning** [54] fine-tunes only the cross-attention layers. **SPT** (sensitivity-aware visual parameter-efficient fine-tuning) [66] first identifies the sensitive parameters measured by the loss reduction when being tuned. This sensitivity is calculated using a first-order Taylor expansion, derived from a single forward and backward pass before fine-tuning in oneshot. Next, SPT finds the weight matrices whose number of sensitive parameters exceeds a pre-defined threshold, and then applies a selected PEFT technique (e.g., LoRA and Adapter) to these targeted weights to achieve structural tuning.

C. Reparameterized PEFT

Reparameterization stands for equivalently transforming a model’s architecture from one to another via transforming its parameters. In the context of PEFT, this often means constructing a low-rank parameterization to achieve the goal of parameter efficiency during training. For inference, the model can be converted to its original weight parameterization, ensuring unchanged inference speed. This procedure is depicted in Figure 4 (c).

Earlier research studies [1] have shown that common pre-trained models exhibit an exceptionally low intrinsic dimensionality. In other words, it is possible to find a low-dimensional reparameterization that is effective for fine-tuning as the entire parameter space. **Intrinsic SAID** [1] is the pioneering work in investigating the intrinsic dimension feature during the fine-tuning of LLMs. However, the most widely recognized reparameterization technique is **LoRA** (Low Rank Adaptation) [76], as shown in Figure 8 (a). For a given pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, LoRA introduces two trainable weight matrices, $W_{up} \in \mathbb{R}^{d \times r}$ and $W_{down} \in \mathbb{R}^{r \times k}$ where the rank $r \ll \min(d, k)$, operating in parallel to W_0 . Let h_{in} represent the input. Under normal conditions, the output through W_0 is $h_{out} = W_0 h_{in}$. Instead, LoRA modifies

this output by introducing an incremental update ΔW that encapsulates task-specific knowledge:

$$h_{out} = W_0 h_{in} + \frac{\alpha}{r} \Delta W h_{in} = W_0 h_{in} + \frac{\alpha}{r} W_{up} W_{down} h_{in}, \quad (12)$$

where α denotes a scaling factor. At the onset of training, W_{down} is initialized using a random Gaussian distribution, while W_{up} is initialized to zero, ensuring that ΔW initially holds a value of zero. LoRA is straightforward to implement and has been evaluated on models with up to 175 billion parameters. Fig 8 (c) used a single decoder as an example, the frozen and learnable components are highlighted in grey and red, respectively. Once fine-tuning is complete, LoRA’s adaptive weights seamlessly integrate with the pre-trained backbone weights. This integration ensures that LoRA maintains the model’s efficiency, adding no extra burden during inference.

In LoRA training, selecting an appropriate rank has always been a challenging issue. To address this, **DyLoRA** [171], as depicted in Figure 8 (b), trains LoRA module on a range of ranks within a predefined training budget, rather than adhering to a single, fixed rank. Specifically, for a given rank range $R = \{r_{min}, r_{min} + 1, \dots, r_{max}\}$, DyLoRA dynamically chooses a rank $r \in R$ at each iteration of the training process. Consequently, the matrices W_{down} and W_{up} are tailored for the selected rank r , resulting in truncated versions $W_{down \downarrow r} = W_{down}[1 : r, :]$ and $W_{up \downarrow r} = W_{up}[:, 1 : r]$, and the subsequent forward and backward pass during this iteration will be restricted on $W_{down \downarrow r}$ and $W_{up \downarrow r}$ instead of W_{down} and W_{up} . With this dynamic and search-free approach, DyLoRA significantly reduces the training time required to find an optimal and fixed LoRA rank for specific tasks. **AdaLoRA** [217] reformulates the ΔW with a singular value decomposition (SVD), denoted as $\Delta W = P \Lambda Q$, where $P \in \mathbb{R}^{d \times r}$ and $Q \in \mathbb{R}^{r \times k}$ are orthonormal, Λ is a diagonal matrix containing singular values $\{\lambda_i\}_{1 \leq i \leq r}$. All the three weight matrices are made learnable. During training, the singular values are pruned iteratively based on their importance scores, which are constructed from moving average of the magnitude of gradient-weight product. To ensure the orthogonality between P and Q , i.e., $P^T P = Q Q^T = I$, an additional regularizer term is included in the loss:

$$R(P, Q) = \|P^T P - I\|_F^2 + \|Q Q^T - I\|_F^2. \quad (13)$$

This adaptive approach enables the model to dynamically adjust the rank within each LoRA module, effectively managing its parameter counts based on the significance of the weight matrices. However, according to **SoRA** [39], the importance scores used in AdaLoRA is heuristically constructed, which lacks rigorous theoretical motivation. Additionally, both moving average operation and calculation of Eq. 13 introduces extra computation cost during training. To address this, SoRA eliminates the orthogonality premise of P and Q . Instead, a gating unit $g \in \mathbb{R}^r$ between W_{up} and W_{down} is directly applied and optimized:

$$h_{\text{out}} = W_{\text{up}}(g \odot (W_{\text{down}}h_{\text{in}})), \quad (14)$$

where $\odot$ is Hadamard product. The gate g is updated using a variation of proximal gradient iteration for l_1 loss [7], [13], which has a clear mathematical meaning and do not need the heuristic premise. After training, the zeroed-out gate units are pruned by removing the corresponding columns and rows in W_{down} and W_{up} .

Several subsequent studies have aimed to improve LoRA's performance in various aspects. For instance, **Laplace-LoRA** [197] notices that fine-tuned LLMs often exhibit overconfidence. To enhance the calibration of fine-tuned LLMs, Laplace-LoRA utilizes a Bayesian approach, specifically a post-hoc Laplace approximation [127], [5], to the posterior over the LoRA parameters. **LoRA+** [65] proposes to set different learning rates for the LoRA matrices W_{down} and W_{up} , such that $\eta_{\text{up}} = \lambda \eta_{\text{down}}$ with $\lambda > 1$ fixed and tune η_{down} . Both **LoRAHub** [80] and **MOELoRA** [119] draw inspiration from multi-task learning. **LoRAHub** aggregates various LoRA modules trained on different tasks. Given a handful of examples from a new task, LoRAHub can autonomously compose compatible LoRA modules without human intervention via a gradient-free method Shiwa [117]. **MOELoRA** employs a Mixture-of-Experts (MOE) approach to train LoRA in a multi-task setting, resulting in multiple expert LoRA modules. To retrieve parameters for certain task, MOELoRA utilizes a task-motivated gate function that assigns contribution weights to each expert based on the task ID, and the final parameters is calculated through a weighted sum of all experts.

In addition to LoRA, several other reparameterization techniques are emerging with significant potential. For instance, **Compacter** [91] introduces a light-weight adapter modules by parameterizing the W_{down} and W_{up} as $W = \sum_{i=1}^n A_i \otimes B_i$, where $A_i \in \mathbb{R}^{n \times n}$, $B_i \in \mathbb{R}^{\frac{r}{n} \times \frac{d}{n}}$, and $\otimes$ denotes the Kronecker product. They further decrease the parameter count by designating A_i as shared parameters and reparameterizing B_i using the product of two low-rank matrices, effectively reducing the parameter complexity from $\mathcal{O}(rd)$ to $\mathcal{O}(r+d)$. Related studies, such as **KronA** [42] and **KAdaptation** [71], also employ the Kronecker product to reparameterize adapter weights, aiming to achieve parameter reduction. **HiWi** [111] proposes an adapter fine-tuning method that applies an adapter directly to pre-trained parameters instead of hidden representations as:

$$W' = W + \sigma(WW_{\text{down}})W_{\text{up}}, \quad (15)$$

where W denotes the weights or biases within the Transformer block's feed-forward layer. Notably, during inference, this

method computes W' in advance, ensuring that the model's inference latency remains on par with that of traditional full fine-tuning. **DoRA** (Weight-Decomposed Low-Rank Adaptation) [120] presents a novel approach as illustrated in Figure 8 (c) by decomposing model weights $W_0 \in \mathbb{R}^{d \times k}$ into magnitude and direction as follows:

$$W_0 = m \frac{V}{\|V\|_c} = \|W_0\|_c \frac{W_0}{\|W_0\|_c}, \quad (16)$$

where $m \in \mathbb{R}^{1 \times k}$ is the magnitude vector, $V \in \mathbb{R}^{d \times k}$ is the directional matrix, with $\|\cdot\|_c$ being the vector-wise norm of a matrix across each column. Subsequently, DoRA adopts a unique fine-tuning strategy for m and V . While both are tunable, only V undergoes LoRA reparameterization, defined as:

$$W' = m \frac{V + \underline{\Delta V}}{\|V + \underline{\Delta V}\|_c} = m \frac{W_0 + \underline{W_{\text{up}}W_{\text{down}}}}{\|W_0 + \underline{W_{\text{up}}W_{\text{down}}}\|_c}, \quad (17)$$

where ΔV is the incremental directional update learned by LoRA, and the underlined parameters denote the trainable parameters. Through this methodology, DoRA consistently outperforms LoRA across various tasks and models, demonstrating its superiority.

D. Hybrid PEFT

The efficacy of various PEFT methods can significantly differ across different tasks. As a result, numerous studies aim to either combine the advantages of diverse PEFT approaches or seek to establish a unified perspective by analyzing the similarities among these methods. For instance, **UniPELT**[133] integrates LoRA, prefix-tuning, and adapters into each Transformer block. To control which PEFT submodules should be activated, they also introduce a gating mechanism. This mechanism consists of three small FFNs that each produce a scalar value $\mathcal{G} \in (0, 1)$, which is then applied to the LoRA, prefix, and adapter matrices, respectively. Across various setups, UniPELT has consistently shown improvements in accuracy ranging from 1% to 4%. **S4** [16] explores design spaces for several PEFT methods (i.e., Adapter (A), Prefix (P), BitFit (B), and LoRA (L)) to uncover underlying design patterns. After a series experiments, their findings include: (1) Applying the spindle grouping partitioning for Transformer layers, which results in four layer groups G_i for $i \in \{1 \dots 4\}$. Layers in one group have similar behaviors together, which means should be apply similar PEFT strategies. (2) Allocating the number of trainable parameters to layers uniformly. (3) Tuning all the groups. (4) Assigning different PEFT strategies in different group. The resulting design space that has the best performance is:

$$G_1 : (A, L), G_2 : (A, P), G_3 : (A, P, B), G_4 : (P, B, L)$$

MAM Adapter[67] explores the intrinsic similarity between three additive PEFT methods: adapters, prefix-tuning, and LoRA, which leads to the development of three variants: *Parallel Adapter*, which places adapter layers alongside specific layers (SA or FFN) instead of after them; *Multi-head Parallel Adapter*, which divides the parallel adapter into multiple